

Tarea 1: mishell, una shell simple en C

Sistemas Operativos 2026, Sección 2, Grupo 8

Integrantes: Joaquín Gustavo Adaúy Castillo, Isaac Amadeus Nelson Castro Villalobos,
Diego Felipe Fuentes Conejeros, Ignacio Esteban Placencia Palma

Repositorio: <https://github.com/SatonakiV/Tarea-Shell-S0>

1. Introducción

mishell es una shell de texto para Linux escrita en C (POSIX). Muestra un prompt con el directorio actual, lee una línea, la parte en comandos y la ejecuta. Soporta redirecciones (<, >, >>), pipes de largo arbitrario, background con &, los built-ins cd, exit, jobs y pmon (un mini top que lee /proc), y no se cae con Ctrl+C ni Ctrl+\. Se compila con make (sin advertencias con -Wall -Wextra -std=gnu11) y se ejecuta con ./mishell (ver README.md). No hicimos bonus.

2. Arquitectura general

2.1 Módulos

Tabla 1. Módulos del proyecto y su responsabilidad.

Archivo	Qué hace
shell.c	Ciclo principal, prompt, getline(), built-ins y señales de la shell. main.c solo llama a run_shell().
parser.c	Tokenizador propio: palabras y operadores (, <, >, >>, &, también pegados como echo hola>out), comillas y \, validación de sintaxis.
executor.c	fork(), grupos de procesos, pipes, dup2(), redirecciones y execvp().
jobs.c	Tabla de jobs, handler de SIGCHLD y espera del foreground.
pmon.c	Lectura de /proc, cálculo de %CPU y refresco con alarm().

2.2 Ciclo principal

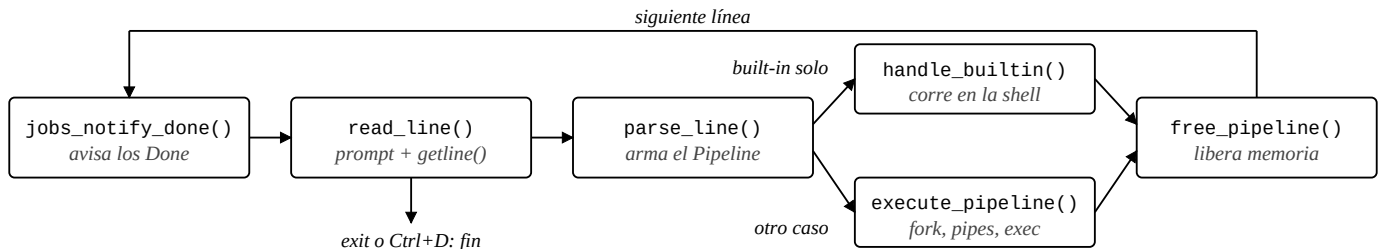


Figura 1. Ciclo principal de run_shell().

Un built-in corre en la shell solo si viene sin pipe ni &; lo demás va a execute_pipeline().

2.3 Estructuras de datos

- **Pipeline:** arreglo de Command, cantidad y flag background. Cada Command trae su argv terminado en NULL y sus Redirection (tipo + archivo). El parser reserva todo y run_shell() lo libera con free_pipeline().
- **Tabla de jobs:** arreglo estático jobs[64] en jobs.c. Cada Job guarda id, arreglo de PIDs del pipeline, pending (hijos que faltan), state (JOB_RUNNING o JOB_DONE, tipo volatile sig_atomic_t), código de salida del último comando, flag background y la línea original para jobs y Done.

2.4 Decisiones de diseño

- Los pipelines en foreground *también* se registran como job. Así el handler de SIGCHLD es el único que llama a waitpid() y el foreground solo espera a que su job pase a JOB_DONE: no hay dos lugares peleando por los mismos hijos.
- Cada pipeline va en su propio grupo de procesos, y Ctrl+C lo reenvía la shell (ver 4.2).
- Los built-ins aceptan redirecciones (jobs > out.txt): se respalda el FD con dup(), se redirige, se ejecuta y se restaura.

3. Pipeline de N comandos

execute_pipeline() crea **los N-1 pipes antes de forkear**, hace fflush(NULL) (para que los hijos no hereden y repitan buffers de stdio) y bloquea SIGCHLD. Después crea un hijo por comando. El hijo i:

1. Entra al grupo del pipeline con `setpgid()` (el primero lo crea con su PID; el padre repite la llamada para que no importe quién corre antes), desbloquea `SIGCHLD` y deja `SIGINT` y `SIGQUIT` en `SIG_DFL`.
2. Hace `dup2(pipes[i-1][0], 0)` si no es el primero y `dup2(pipes[i][1], 1)` si no es el último.
3. Aplica sus redirecciones con `open() + dup2() + close()` (`O_RDONLY`, `O_WRONLY|O_CREAT|O_TRUNC` u `O_WRONLY|O_CREAT|O_APPEND`, modo 0644). Van después de los pipes, así un `>` explícito le gana al pipe.
4. Cierra los 2(N-1) extremos originales con `close_all_pipes()`: el 0 y el 1 ya apuntan a donde deben.
5. Llama a `execvp()`. Si falla imprime el error y hace `_exit(127)` (no existe) o `_exit(126)` (no ejecutable); un `open()` fallido hace `_exit(1)`. Es `_exit` y no `exit` para no vaciar buffers heredados del padre.

La shell cierra todos sus extremos apenas termina el ciclo de `fork()`. Con `cmd1 | cmd2 | cmd3` (pipes `p0` y `p1`) queda así:

Tabla 2. Descriptores de cada proceso de `cmd1 | cmd2 | cmd3` justo antes del `exec`.

Proceso	FD 0	FD 1	Extremos de pipe que conserva
cmd1	terminal	<code>p0[1]</code>	ninguno (ya duplicó el que usa)
cmd2	<code>p0[0]</code>	<code>p1[1]</code>	ninguno
cmd3	<code>p1[0]</code>	terminal	ninguno
shell	terminal	terminal	ninguno (los cierra tras los <code>fork</code>)

Si un `fork()` falla a mitad de camino, la shell manda `SIGTERM` al grupo, espera a los hijos que alcanzó a crear (no quedan colgados ni zombies) y termina, como pide R1.

4. Manejo de señales

4.1 SIGCHLD

Handler con `sigaction()`. Hace `while ((pid = waitpid(-1, &st, WNOHANG)) > 0)`, busca el job de ese PID, descuenta pending, guarda el código si es el último comando y marca `JOB_DONE` cuando pending llega a 0. Respalda `errno` y no imprime nada (`printf` no es `async-signal-safe`): el `[n]+ Done` lo muestra `jobs_notify_done()`. Va *sin* `SA_RESTART`: si un background termina con la shell en el prompt, `getline()` vuelve con `EINTR` y el aviso sale al tiro. `SIGCHLD` queda bloqueado desde antes del primer `fork()` hasta que el job está registrado, así un hijo que muere muy rápido igual encuentra su job. El foreground espera con `while (state != JOB_DONE) sigsuspend(&old_mask)`, que desbloquea y duerme de forma atómica (sin la carrera de `pause()`).

4.2 SIGINT y SIGQUIT

La shell atrapa las dos (sin `SA_RESTART`, para que `getline()` vuelva con `EINTR` y el prompt se redibuje). Cada handler lee `foreground_pgid` (`volatile sig_atomic_t`, 0 si no hay foreground) y le reenvía la señal a ese grupo con `kill(-pgid, sig)` (`async-signal-safe`). En el prompt no hace nada. Los hijos las dejan en `SIG_DFL` y mueren con ellas.

Como no usamos `tcsetpgrp()`, la terminal sigue apuntando al grupo de la shell y `Ctrl+C` solo le llega a ella. Los background están en otro grupo y nadie les reenvía nada, así que sobreviven aunque instalen su propio handler.

4.3 SIGTSTP

No se maneja (no hicimos el bonus de `Ctrl+Z` con `fg/bg`). Por lo mismo, `Ctrl+Z` detiene solo a `mishell` (el foreground sigue corriendo) y la shell desde donde se lanzó la recupera con `fg`.

5. pmon

5.1 Campos de /proc

`pmon [segundos]` (2 por defecto) toma de `jobs.c` los PIDs de los background vivos (todos los de cada pipeline) y lee:

- `/proc/[pid]/stat`: nombre entre paréntesis (se busca el último) con `strchr`, porque el nombre puede tener espacios, `state` (campo 3) y `utime` y `stime` (campos 14 y 15, en ticks).
- `/proc/[pid]/status`: la línea `VmRSS`: en kB.

5.2 %CPU y refresco

`%CPU` = $(\text{delta de utime+stime} / \text{sysconf}(_SC_CLK_TCK)) / (\text{delta de tiempo real}) \times 100$, con el tiempo real medido con `clock_gettime(CLOCK_MONOTONIC)` entre dos muestras seguidas. Para refrescar se usa `alarm(intervalo)` y un handler de `SIGALRM` que solo pone `actualizar = 1` (`volatile sig_atomic_t`); el ciclo espera con `sigsuspend()`. Mientras corre, `pmon` instala su propio handler de `SIGINT` que marca `salir_pmon`; al salir hace `alarm(0)` y restaura los handlers anteriores de `SIGINT` y `SIGALRM`, así `Ctrl+C` vuelve al prompt sin cerrar la shell. Si un proceso termina, su muestra se marca inválida y deja de listarse (ver pregunta 5).

6. Pruebas realizadas

- **Ejecutor** (`make test`): 21 casos contra `execute_pipeline()`: códigos 127 y 126, muerte por señal (128+15), `>`, `>>`, `<`, combinadas, `open()` fallido, pipes de 2, 3 y 4, `yes | head -5`, comando intermedio fallido y código del último. Todos pasan.
- **pmon** (`tests/tests_pmon/`): lectura de `/proc` y %CPU contra un PID real (yes da cerca de 99%).
- **Shell completa**: sesiones en una pseudo-terminal (PTY) con la sesión de ejemplo de la pauta, errores de sintaxis, `cd`, `exit 300` (sale con 44) y `Ctrl+D`. Con `sleep 30 &`, `yes > /dev/null &`, `sleep 2 | sleep 3 &` y `python3` en background probamos `jobs`, `pmon 1`, `Ctrl+C` (prompt, `sleep 100` y `pmon`) y `Ctrl+\` (`sleep 100`): la shell y los background sobreviven.
- **Zombies y descriptores**: un `sleep 1 &` que termina durante un `sleep 2` no deja hijos Z (`ps -ppid` desde afuera), y `/proc/<pid>/fd` de la shell muestra solo 0, 1 y 2 tras pipes y redirecciones fallidas.
- **fork() fallido, sanitizers y estrés**: con `ulimit -u` bajo, un pipeline de 12 `sleep` hace fallar un `fork()` y la shell avisa y termina sin dejar ningún `sleep` vivo. Con `make san` (ASan y UBSan) las sesiones no reportan nada, y `70 sleep 6 &` seguidos dan 64 jobs y 6 rechazados con aviso, todos sin zombies.

7. Limitaciones conocidas

Algunas cosas fallan dentro de lo que pide la pauta y otras simplemente quedaron fuera porque no se pedían.

- **La terminal no se le entrega al foreground** (falta `tcsetpgrp()`). Es lo más serio: un foreground que lee de la terminal (`cat` solo, `sort` sin archivo, `python3`, `less`) recibe `SIGTTIN` o `SIGTTOU` y queda detenido (estado T). `Ctrl+C` no lo destraba y la shell se queda esperando hasta un `kill -9` desde afuera. Un background que lee de la terminal también se detiene y `jobs` lo sigue mostrando "Ejecutando".
- **Detalles dentro de lo pedido**. La tabla tiene 64 jobs (del background 65 en adelante la shell avisa y lo mata con `SIGTERM`); los built-ins solo andan sueltos (en `jobs | wc -l` la shell dice que `jobs` no existe); `pmon` muestra la letra de estado y el nombre corto, no limpia la pantalla y la primera tabla sale tras el primer intervalo; el Done repite el `&` y puede salir pegado al prompt; los ids de job saltan (los foreground también usan uno), y el binario `mishell` (viejo) está subido al repo.
- **Sin separadores**. No hay `;`, `&&`, `||` ni subshells (`echo uno; echo dos` imprime `uno; echo dos`).
- **Solo stdin y stdout**. No hay `2>`, `2>&1` ni `<<`: en `ls x 2> err`, el 2 le llega a `ls` como argumento.
- **Sin expansiones**. `$HOME`, `~` y los comodines llegan tal cual (`echo *.c` imprime `*.c` y `cd ~` falla), y no hay `$(...)`. Tampoco hay historial, variables propias ni `Ctrl+Z` con `fg/bg`, que la pauta deja como bonus.
- **exit ignora los background**. Si quedan jobs vivos no avisa ni los termina: siguen corriendo huérfanos.

8. Preguntas de reflexión

1. **¿Por qué `cd` y `exit` son internos?** Cada proceso tiene su propio directorio de trabajo. Si `cd` corriera en un hijo, cambiaría el directorio del hijo, que muere enseguida, y la shell seguiría donde estaba. Con `exit` pasa lo mismo: terminaría el hijo.
2. **¿Qué pasa si no se cierran los pipes?** Si en `cmd1 | cmd2 | cmd3` la shell se queda con `p1[1]` abierto, `cmd3` (digamos `wc -l`) nunca recibe EOF porque para el kernel sigue habiendo un escritor. `wc` se queda bloqueado en `read()` y el pipeline se cuelga.
3. **¿Por qué `waitpid` en ciclo con `WNOHANG`?** Las señales no se encolan: si dos hijos mueren casi juntos (típico en un pipeline) puede llegar un solo `SIGCHLD`. Con una llamada se recoge uno y el otro queda zombie. El ciclo recoge todos los que hay y `WNOHANG` evita que el handler se bloquee cuando quedan hijos vivos.
4. **¿Y si no se restaura `SIGINT` en el hijo?** En nuestro caso casi no se notaría: la shell *atrapa* `SIGINT` y `SIGQUIT`, y `exec` resetea a `SIG_DFL` toda señal atrapada. La diferencia grande aparece si la shell las *ignora* (`SIG_IGN`, como dice la pauta al pie de la letra): `SIG_IGN` sobrevive al `exec` y el foreground quedaría inmune a `Ctrl+C` y `Ctrl+\`. Además, sin `set_child_signals()` un `Ctrl+C` entre `fork()` y `exec` lo atraparía el handler copiado de la shell y se perdería.
5. **¿Qué pasa si un proceso muere entre dos lecturas?** El handler de `SIGCHLD` lo recoge y `/proc/[pid]` desaparece, así que `fopen()` falla (o falta `VmRSS`, que un zombie no tiene). `tomar_muestra()` devuelve -1, la muestra se marca con `pid = -1`, el proceso no sale en la siguiente tabla y `pmon` sigue normal.

9. Conclusión

Lo central funciona: pipes sin FD filtrados, redirecciones, background sin zombies, `fork()` fallido bien cerrado y señales reenviadas por grupo. Lo que más pesa es no entregarle la terminal al foreground: todo comando que la lea se detiene. Tener un solo recolector de hijos (el handler de `SIGCHLD`) simplificó mucho la sincronización.